



Pandas Playground - Documentación del Proyecto

Tabla de Contenidos

1. [Descripción General](#)
 2. [Características](#)
 3. [Estructura del Proyecto](#)
 4. [Arquitectura Técnica](#)
 5. [Lecciones](#)
 6. [Sistema de Validación](#)
 7. [Guía Informativa](#)
 8. [Diseño Responsive](#)
 9. [Funcionalidades Interactivas](#)
 10. [Almacenamiento Local](#)
 11. [Dependencias Externas](#)
 12. [Compatibilidad](#)
 13. [Solución de Problemas](#)
-

Descripción General

Pandas Playground es una aplicación web interactiva para aprender la librería pandas de Python. Está diseñada como una plataforma de aprendizaje tipo freeCodeCamp, donde los usuarios pueden escribir código Python directamente en el navegador y ejecutarlo contra tests automáticos.

Objetivo

Enseñar los conceptos fundamentales de pandas de manera práctica e interactiva, permitiendo al estudiante aprender haciendo (learning by doing).

Público Objetivo

- Principiantes en análisis de datos con Python
 - Estudiantes que quieren aprender pandas
 - Desarrolladores que buscan una referencia interactiva
-

Características

✓ Editor de Código Integrado

- Monaco Editor (el mismo de VS Code)
- Syntax highlighting para Python
- Números de línea
- Autocompletado básico
- Indentación automática

✓ Ejecución en el Navegador

- Pyodide (Python compilado a WebAssembly)
- pandas pre-cargado
- No requiere servidor backend
- Ejecución instantánea después de la carga inicial

✓ Sistema de Tests Automáticos

- Múltiples tests por lección
- Validación en tiempo real
- Mensajes de error descriptivos
- Contador de tests pasados

✓ Modal de Errores

- Popup emergente cuando hay errores de sintaxis
- Muestra el mensaje exacto de error de Python
- Cierra con clic fuera, botón o tecla Escape

✓ Soluciones

- Botón "💡 Solución" en cada lección
- Confirma antes de reemplazar el código
- Muestra la solución completa correcta

✓ Guía Informativa

- Modal con documentación completa
- 12 funciones de pandas documentadas
- Ejemplos de código diferentes a las soluciones
- Accesible desde cualquier lección

✓ **Diseño Responsive**

- Adaptado a desktop, tablets y móviles
- Layout split en pantallas grandes
- Layout vertical en pantallas pequeñas
- Modales adaptativos

✓ **Progreso Persistente**

- Guarda lecciones completadas en localStorage
- Indicadores visuales de progreso
- Puede continuar donde quedó

Estructura del Proyecto

```
Default Project/  
├─ index_html          # Archivo principal (HTML/CSS/JS)
```

Archivos Principales

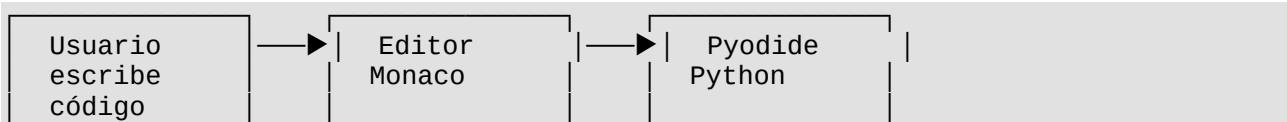
Archivo	Descripción	Tamaño Aprox.
index.html	Aplicación interactiva completa	~50KB

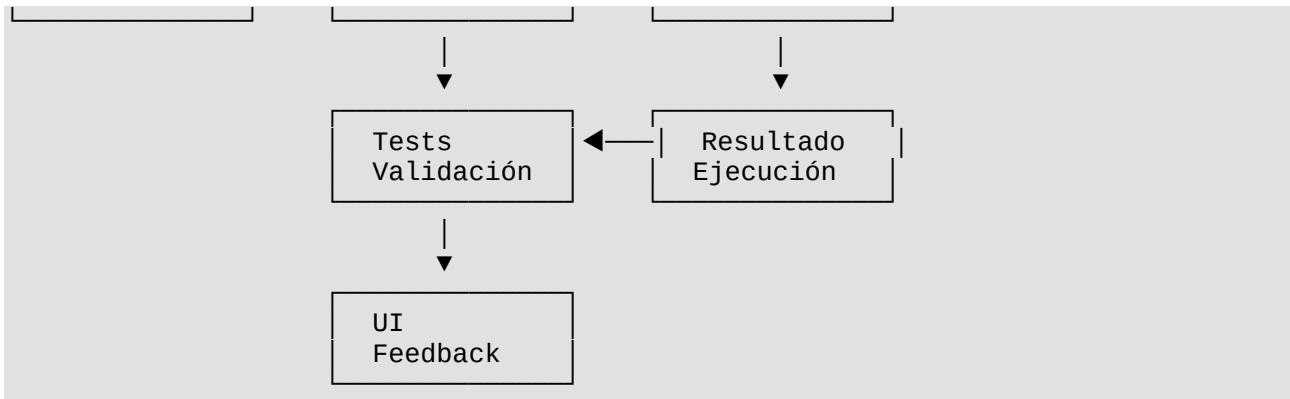
Arquitectura Técnica

Stack Tecnológico

Frontend	
HTML5	- Estructura semántica
CSS3	- Estilos y diseño responsive
JavaScript	- Lógica de la aplicación
Dependencias CDN	
Pyodide v0.25	- Python en WebAssembly
Monaco Editor	- Editor de código
pandas	- Librería de análisis de datos

Flujo de Ejecución





Ciclo de Vida de una Lección

1. **Carga:** Se muestra el código semilla en el editor
2. **Edición:** El usuario modifica el código
3. **Ejecución:** Pyodide ejecuta el código
4. **Validación:** Los tests verifican el resultado
5. **Feedback:** Se muestran los resultados (pass/fail)
6. **Progreso:** Si pasa, se marca como completada

Lecciones

Estructura de Datos

Cada lección es un objeto JavaScript con la siguiente estructura:

```
{
    id: 1,                // Número de lección
    title: "Nombre de la Lección", // Título visible
    instructions: `HTML`,    // Instrucciones en HTML
    seedCode: `Python`,      // Código inicial
    solution: `Python`,      // Código solución
    tests: [                // Array de tests
        {
            name: "Nombre del test",
            code: `Código de setup`,
            validation: `Expresión booleana`,
            error: "Mensaje de error"
        }
    ]
}
```

Lecciones Implementadas

#	Título	Concepto Principal	Tests
1	Importar Pandas	<code>import pandas as pd</code>	1
2	Crear una Serie	<code>pd.Series()</code>	3

#	Título	Concepto Principal	Tests
3	Crear un DataFrame	<code>pd.DataFrame()</code>	4
4	Leer un CSV	<code>pd.read_csv()</code>	4
5	Ver los primeros datos	<code>df.head()</code>	3
6	Filtrar datos	Indexación booleana	4
7	Estadísticas básicas	<code>.mean()</code>	3
8	Ordenar datos	<code>.sort_values()</code>	3
9	Agregar columna	Operaciones columnas	3
10	Agrupar datos	<code>.groupby()</code>	4
11	Valores faltantes	<code>.fillna()</code>	3
12	Eliminar duplicados	<code>.drop_duplicates()</code>	3
13	Exportar a CSV	<code>.to_csv()</code>	2
14	Proyecto Final	Integración	3

Total: 14 lecciones, 43 tests

Sistema de Validación

Cómo Funciona

```
// 1. Se ejecuta el código del usuario
pyodide.runPython(userCode);

// 2. Se ejecuta la validación del test
const result = pyodide.runPython(test.validation);

// 3. Si result es True, el test pasa
if (result) {
    passed++;
} else {
    // Mostrar error
}
```

Tipos de Validación

Tipo	Ejemplo	Descripción
Existencia	<code>'df' in dir()</code>	Verifica que la variable exista
Tipo	<code>isinstance(df, pd.DataFrame)</code>	Verifica el tipo de dato
Valor	<code>len(df) == 4</code>	Verifica un valor específico
Contenido	<code>list(df.columns) == [...]</code>	Verifica el contenido
Complejo	<code>set(df['col']) ==</code>	Validaciones avanzadas

Tipo	Ejemplo	Descripción
	{'a', 'b'}	

Manejo de Errores

```
try {
  pyodide.runPython(userCode);
} catch (error) {
  // Detectar tipo de error
  if (errorMsg.includes('SyntaxError')) {
    showModal(errorMsg); // Modal para errores de sintaxis
  }
  if (errorMsg.includes('NameError')) {
    showModal(errorMsg); // Modal para nombres incorrectos
  }
  if (errorMsg.includes('TypeError')) {
    showModal(errorMsg); // Modal para tipos incorrectos
  }
}
```

Guía Informativa

Contenido Documentado

Función	Categoría	Descripción
import pandas as pd	Importación	Importar la librería
pd.Series()	Estructuras	Crear arrays etiquetados
pd.DataFrame()	Estructuras	Crear tablas de datos
pd.read_csv()	I/O	Leer archivos CSV
df.head()	Exploración	Ver primeras filas
Filtrado booleano	Selección	Seleccionar por condiciones
Operaciones estadísticas	Análisis	mean, sum, max, min
df.sort_values()	Ordenamiento	Ordenar datos
Crear columnas	Transformación	Operaciones entre columnas
df.groupby()	Agregación	Agrupar y agregar
dropna/fillna	Limpieza	Manejar NaN
df.drop_duplicates()	Limpieza	Eliminar duplicados
df.to_csv()	I/O	Exportar a CSV

Diseño de Ejemplos

Los ejemplos de la guía están diseñados para:

- **No dar soluciones directas:** Usan datos diferentes a los ejercicios

- **Ser ilustrativos:** Muestran el caso de uso más común
 - **Incluir variaciones:** Múltiples formas de usar la función
-

Diseño Responsive

Breakpoints

```
/* Tablets */
@media (max-width: 900px) {
  /* Layout vertical */
  /* Paneles apilados */
}

/* Móviles */
@media (max-width: 600px) {
  /* Tamaños reducidos */
  /* Padding compacto */
}
```

Comportamiento por Dispositivo

Dispositivo	Ancho	Layout
Desktop	>900px	Split horizontal (50/50)
Tablet	600-900px	Vertical (instrucciones arriba)
Móvil	<600px	Vertical compacto

Elementos Adaptivos

- **Header:** Se apila en móviles
 - **Step navigation:** Se mantiene horizontal con scroll
 - **Editor:** Ocupa el espacio disponible
 - **Modales:** Se ajustan al 90% del ancho
 - **Guía:** Scroll interno con contenido adaptado
-

Funcionalidades Interactivas

1. Ejecutar Tests (▶)

```
function runTests() {
  // 1. Validar sintaxis
  // 2. Ejecutar código por cada test
  // 3. Validar resultado
  // 4. Mostrar feedback
  // 5. Actualizar progreso si pasa
}
```

2. Reiniciar Código (🔄)

```
function resetCode() {  
  // 1. Mostrar confirmación  
  // 2. Restaurar código semilla  
  // 3. Limpiar resultados  
}
```

3. Mostrar Solución (💡)

```
function showSolution() {  
  // 1. Verificar que exista solución  
  // 2. Mostrar confirmación  
  // 3. Reemplazar código en editor  
}
```

4. Guía Informativa (📖)

```
function openGuide() {  
  // 1. Mostrar modal con documentación  
  // 2. Mantener scroll independiente  
}
```

5. Modal de Errores (⚠️)

```
function showModal(errorMessage) {  
  // 1. Mostrar icono de advertencia  
  // 2. Mostrar mensaje genérico  
  // 3. Mostrar detalles del error  
  // 4. Permitir cerrar con botón/Escape/clic externo  
}
```

6. Navegación por Pasos

```
function loadLesson(index) {  
  // 1. Actualizar lección actual  
  // 2. Cargar código semilla  
  // 3. Limpiar tests  
  // 4. Actualizar navegación visual  
}
```

Almacenamiento Local

Datos Guardados

Clave	Tipo	Descripción
pandasCompleted	Array	IDs de lecciones completadas

Estructura

```
// Ejemplo  
localStorage.setItem('pandasCompleted', JSON.stringify([0, 1, 2, 3]));  
  
// Lectura
```



```
const completed = JSON.parse(localStorage.getItem('pandasCompleted')) || [];
```

Persistencia

- Se guarda automáticamente al pasar un test
 - Se carga al iniciar la aplicación
 - Se mantiene entre sesiones del navegador
 - Se puede resetear borrando localStorage
-

Dependencias Externas

Pyodide v0.25.0

```
<script src="https://cdn.jsdelivr.net/pyodide/v0.25.0/full/pyodide.js"></script>
```

- **Función:** Ejecutar Python en el navegador
- **Tamaño:** ~6MB (se descarga una vez)
- **Paquetes:** pandas pre-cargado
- **Tiempo de carga:** 3-5 segundos (primera vez)

Monaco Editor v0.45.0

```
<script  
src="https://cdn.jsdelivr.net/npm/monaco-editor@0.45.0/min/vs/loader.js"></  
script>
```

- **Función:** Editor de código
 - **Características:** Syntax highlighting, autocompletado, números de línea
 - **Tamaño:** ~2MB (carga bajo demanda)
-

Compatibilidad

Navegadores Soportados

Navegador	Versión Mínima	Estado
Chrome	80+	✓ Completo
Firefox	78+	✓ Completo
Safari	14+	✓ Completo
Edge	80+	✓ Completo

Requisitos

- **JavaScript:** Habilitado
 - **WebAssembly:** Soportado
 - **Conexión:** Internet (para carga inicial de dependencias)
 - **Almacenamiento:** ~50KB para localStorage
-

Solución de Problemas

Pyodide no carga

Síntoma: Se queda en "Cargando entorno de Python..."

Soluciones:

1. Verificar conexión a internet
2. Desactivar bloqueadores de anuncios
3. Limpiar caché del navegador
4. Probar en otro navegador

Tests no pasan

Síntoma: El código parece correcto pero falla

Posibles causas:

1. Nombre de variable incorrecto
2. Tipo de dato incorrecto
3. Valor esperado diferente
4. Error de indentación

Solución: Usar el botón "Solución" para ver la respuesta correcta

Modal no aparece

Síntoma: Los errores se muestran solo en el panel inferior

Causa: El error no es de sintaxis, nombre o tipo

Solución: Revisar los tests en el panel de resultados

Diseño roto en móvil

Síntoma: Los paneles se superponen

Solución:

1. Rotar el dispositivo

2. Recargar la página
 3. Usar en landscape
-

Optimizaciones Implementadas

Carga

- Dependencias desde CDN (cached)
- Pyodide se carga de forma asíncrona
- Monaco Editor carga bajo demanda

Rendimiento

- Re-render solo del componente afectado
- Debounce en eventos de resize
- Lazy loading de contenido

UX

- Feedback inmediato del usuario
 - Confirmación antes de acciones destructivas
 - Atajos de teclado (Escape para cerrar modales)
-

Futuras Mejoras

Planeadas

- ☐ Más lecciones (15-20 total)
- ☐ Modo oscuro
- ☐ Exportar progreso
- ☐ Compartir código
- ☐ Tests personalizados

Posibles

- ☐ Soporte para NumPy
- ☐ Soporte para Matplotlib
- ☐ Modo desafío (tiempo límite)
- ☐ Tabla de clasificación
- ☐ Certificado de completado

Créditos

Tecnologías

- **Pyodide:** <https://pyodide.org/>
- **Monaco Editor:** <https://microsoft.github.io/monaco-editor/>
- **pandas:** <https://pandas.pydata.org/>

Diseño

- Inspirado en freeCodeCamp (<https://www.freecodecamp.org/>)
- Iconos: Emojis nativos del navegador
- Colores: Paleta personalizada

Licencia

Este proyecto es educativo y de uso libre.

Última actualización: Agosto 2026 Versión: 1.0.0